# Tema 2:
# Programació multifil

Setmana 2. Programació multifil

# Introducció

L'execució de tasques en paral·lel optimitza el temps d'utilització dels recursos de sistema.
Exemple: quan un procés està esperant la finalització d'una operació d'E/S, altres processos poden aprofitar el processador de sistema que en aquest moment no s'usa.

**La programació paral·lela és molt més complexa que la convencional.**

# Processos i fils

**Processos**

- Disposen del seu propi espai de memòria.
- Es comuniquen a través de pipes o sockets.

**Fils**

- Execucions simultànies dins d'un mateix procés
  (podem considerar-los com processos lleugers)
- Espai de memòria compartit

# Fils (Threads)

Els fils no poden executar-se en solitari. Necessiten un procés.

Utilitat fils: Realitzar programes que tinguen que realitzar diverses tasques simultàniament.

**Exemple: en una fàbrica un programa controla els sensors i cada sensor és un fil.**

**Exemple: processador de textos, 1 fil corregeix ortografia, 1 fil guarda fitxer automàticament.**

# Fils en Java

Quan s'inicia un programa en Java, la màquina virtual crea un fil principal.

- El fil principal s'encarregarà d'invocar el mètode main de la classe que es comença a executar.
- El fil principal acaba quan s'acaba d'executar el mètode main.
- Si el fil principal crea altres fils, éstos començaran la seva execució de forma concurrent.
- Només quan no queda cap fil actiu, és quan s'acaba el fil principal i, per tant, el programa.

# Thread

La classe principal per aconseguir concurrència en Java és la classe Thread.

Disposa d'un mètode **start()** que inicialitza el nou fil i executa el seu mètode **run().**

# Creació de fils: 2 alternatives

1. Heretant de la classe Thread.
2. Implementant la interfície Runnable

# Creació de fils. Thread

Esquema general

1. Crear subclasse estenent de la classe Thread.
2. Implementar el mètode run. El codi defineix el que farà el fil durant la seua execució.
3. Es crea el fil i es crida al seu mètode start, que s'encarrega, entre altres coses, de cridar a run.

# Creació de fils. Thread

Estructura bàsica

estructurabasicafil.java

```
1   //Estructura bàsica fil
2   Class NomFil extends Thread {
3       //atributs, constructors i métodes classe
4
5       public void run() {
6           //accions que ha de fer el fil
7       }
8   }
```

# Creació de fils. Thread. T2S2Exemple1

PrimerFil.java

```java
public class PrimerFil extends Thread {

    //Atributs classe
    private int x;

    //Constructor classe
    public PrimerFil (int x)
    {
        this.x=x;
    }//fi constructor

    //Mètode d'execució (run). Accions que fa el fil
    public void run(){
        for (int i=0;i<x;i++)
            System.out.println ("Dins del fil..");
        }//fi run

    //Mètode main
    public static void main(String[] args){
        PrimerFil p=new PrimerFil(10); //cree el fil
        p.start();
    } //fi main

} //fi classe
```

```
[ramon@nowhere Exemples]$ javac PrimerFil.java
[ramon@nowhere Exemples]$ java PrimerFil
Dins del fil..
Dins del fil..
Dins del fil..
Dins del fil..
Dins del fil..
Dins del fil..
Dins del fil..
Dins del fil..
Dins del fil..
Dins del fil..
```

→ Fixa't que s'estén de Thread

→ Després definim propietats de la classe i constructor.

→ Mètode run

# Creació Fil. T2S2Exemple2.java 1/2

FilExemple1.java

```
1  public class FilExemple1 extends Thread {
2      //Atributs ---------------------------------
3      private int c; //comptador fil
4      private int fil;
5
6      //Constructor ------------------------------
7      public FilExemple1 (int fil){
8          this.fil=fil;
9          System.out.println("Creant Fil: " + fil);
10     }//fi constructor
11
12     //Mètode Run -------------------------------
13     public void run(){
14         c=0;
15         while (c<=5){
16             System.out.println ("Fil:"+ fil + " C= " + c);
17             c++;
18         }
19     }//fi run
20
```

Fixa't que s'estén de Thread

Després definim propietats de la classe i el constructor.

Mètode run

# Creació Fil. T2S2Exemple2 2/2

```
21      //Mètode main
22      public static void main(String[] args){
23      FilExemple1 h=null;
24      for (int i=0;i<3;i++){
25          h=new FilExemple1(i+1); //cree fil
26          h.start(); //inicialitzar fil
27          }
28      System.out.println("3 Fils creats...");
29      } //fi main
30
31  } //fi classe
```

```
[ramon@nowhere Exemples]$ javac FilExemple1.java
[ramon@nowhere Exemples]$ java FilExemple1
Creant Fil: 1
Creant Fil: 2
Creant Fil: 3
3 Fils creats...
Fil:2 C= 0
Fil:2 C= 1
Fil:1 C= 0
Fil:1 C= 1
Fil:2 C= 2
Fil:2 C= 3
Fil:1 C= 2
Fil:1 C= 3
Fil:1 C= 4
Fil:1 C= 5
Fil:3 C= 0
Fil:3 C= 1
Fil:3 C= 2
Fil:3 C= 3
Fil:3 C= 4
Fil:3 C= 5
Fil:2 C= 4
Fil:2 C= 5
```

Mètode main: observa com realitza bucle.... i en cada iteració, crea un fil i crida al mètode start per iniciar-lo.

Molt usual en el run del fil tenir un bucle infinit de manera que el fil no acaba tret que no utilitzem el mètode que veurem.

Els fils no s'executen en ordre creació!!!

# Crida a fil des d'una altra classe

En l'exemple anterior, el main està inclòs dins de la classe.

Però podem tenir una classe fil i una classe que crida a la classe fil.

Modifiquem l'exemple:

FilExemple1_V2.java

```
1  public class FilExemple1_V2 extends Thread {
2      //Atributs ---------------------------------
3      private int c; //comptador fil
4      private int fil;
5
6      //Constructor ------------------------------
7      public FilExemple1_V2 (int fil){
8          this.fil=fil;
9          System.out.println("Creant Fil: " + fil);
10     }//fi constructor
11
12     //Mètode Run -------------------------------
13     public void run(){
14         c=0;
15         while (c<=5){
16             System.out.println ("Fil:"+ fil + " C= " + c);
17             c++;
18         }
19     }//fi run
20 } //fi clase
```

# Classe crida al fil

UsaFilExemple1_V2.java

```
1  public class UsaFilExemple1_V2 {
2    public static void main(String[] args){
3          FilExemple1_V2 h=null;
4          for (int i=0;i<3;i++){
5              h=new FilExemple1_V2(i+1); //cree fil
6              h.start(); //iniciar fil
7              //comprovem si este fil està viu
8              System.out.println("status = " + h.isAlive());
9          }
10         System.out.println("3 Fils creats...");
11     } //fi main
12 } //fi classe
13
```

```
[ramon@nowhere Exemples]$ javac FilExemple1_V2.java
[ramon@nowhere Exemples]$ javac UsaFilExemple1_V2.java
[ramon@nowhere Exemples]$ java UsaFilExemple1_V2
Creant Fil: 1
status = true
Creant Fil: 2
status = true
Creant Fil: 3
status = true
3 Fils creats...
Fil:2 C= 0
Fil:1 C= 0
Fil:1 C= 1
Fil:1 C= 2
Fil:3 C= 0
Fil:1 C= 3
Fil:2 C= 1
Fil:1 C= 4
Fil:3 C= 1
Fil:1 C= 5
Fil:2 C= 2
Fil:2 C= 3
Fil:2 C= 4
Fil:2 C= 5
Fil:3 C= 2
Fil:3 C= 3
Fil:3 C= 4
Fil:3 C= 5
```

# Mètodes útils en fils

- **start()** → Fa que el fil comence l'execució.
- **boolean isAlive()** → Comprova si el fil és viu. Retorna True si està viu.
- **join()** → Espera que acabe el fil.
- **getState()** → Retorna l'estat del fil: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED

# Exemple: Mètodes en ús

Creem una classe fil simple (T2S2Exemple4Alive)

FilExemple_Alive.java

```
1  public class FilExemple_Alive extends Thread {
2
3      //Atributs --------------------------------
4      private int c; //comptador fil
5      private int fil;
6
7      //Constructor -----------------------------------
8      public FilExemple_Alive (int fil){
9          this.fil=fil;
10         System.out.println("Creant Fil: " + fil);
11     }//fi constructor
12
13     //Mètode Run ------------------------------------
14     public void run(){
15         c=0;
16         while (c<=5){
17             System.out.println ("Fil:"+ fil + " C= " + c);
18             c++;
19         }
20     }//fi run
21 } //fi classe
```

# Exemple: Mètodes en ús

UsaFil_Alive.java

```
1  import java.lang.*;
2  public class UsaFil_Alive {
3    public static void main(String[] args){
4        FilExemple Alive h=null;
5        h=new FilExemple Alive(1); //cree fil
6
7        //Comprove l'estat abans de cridar a start
8        System.out.println("Abans cridada a start");
9        System.out.println("Està viu? = " + h.isAlive());
10       System.out.println("Estat:" + h.getState());
11
12       System.out.println("Cride a start");
13       h.start(); //inicialitzar fil
14
15       //Comprove l'estat després de cridar a start
16       System.out.println("Estat:" + h.getState());
17       System.out.println("Està viu?? = " + h.isAlive());
18
19       //Cride a join i espere a que acabe
20       try{
21           h.join();
22           } catch (Exception ex){}
23
24
25       //Comprove l'estat després de finalitzar el fil
26       System.out.println("Després de finalitzar fil ");
27       System.out.println("Estat:" + h.getState());
28       System.out.println("Està viu? = " + h.isAlive());
29     } //fi main
30 } //fi classe
31
```

# Exemple: T2S2Exemple4AliveUsa

```
[ramon@nowhere Exemples]$ javac UsaFil_Alive.java
[ramon@nowhere Exemples]$ java UsaFil_Alive
Creant Fil: 1
Abans cridada a start
Està viu? = false
Estat:NEW
Cride a start
Estat:RUNNABLE
Està viu?? = true
Fil:1 C= 0
Fil:1 C= 1
Fil:1 C= 2
Fil:1 C= 3
Fil:1 C= 4
Fil:1 C= 5
Després de finalitzar fil
Estat:TERMINATED
Està viu? = false
```

# Mètodes útils en fils

- **sleep(long mils)** → Fa que el fil en execució passe a dormir durant els mil·lisegons especificats
- **toString()** → Retorna una representació en format cadena d'aquest fil, incloent nom del fil, la prioritat i el grup de fils.
- **getId()** → Retorna l'identificador del fil

# Exemple. Mètodes en ús: getId i toString

UsaFil_toString.java

```
1   import java.lang.*;
2   public class UsaFil_toString {
3     public static void main(String[] args){
4           FilExemple Alive h=null;
5           h=new FilExemple Alive(1); //cree fil
6
7           h.start(); //iniciar fil
8
9           //Comprove estat després de cridar a start
10          System.out.println("Informació id fil:" + h.getId());
11          System.out.println("Informació fil = " + h.toString());
12    } //fi main
13  } //fi clase
14
```

**Atenció: l'eixida pot variar d'una execució a una altra**

Executat pel fil principal: id del fil creat

```
[ramon@nowhere Exemples]$ javac UsaFil_toString.java
[ramon@nowhere Exemples]$ java UsaFil_toString
Creant Fil: 1
Informació id fil:11
Fil:1 C= 0
Fil:1 C= 1
Fil:1 C= 2
Fil:1 C= 3
Fil:1 C= 4
Fil:1 C= 5
Informació fil = Thread[Thread-0,5,main]
```

Fil creat

Executat pel fil principal: info del fil creat Thread [nom fil, prioritat, grup de fils]

# Exemple. Mètodes en ús: sleep

UsaFil_Sleep.java

```
1  public class UsaFil_Sleep {
2    public static void main(String[] args){
3         FilExemple Alive h=null;
4         h=new FilExemple Alive(1); //cree fil
5         h.start(); //iniciar fil
6
7         try {
8              //Pausa 4 segons
9              h.sleep(4000);
10             } catch (InterruptedException e) {
11                                  }
12     } //fi main
13 } //fi classe
14
```

# Mètodes útils en fils

- **getName()** → Retorna nom del fil
- **setName(String name)** → Canvia el nom a name.
- **getPriority()** → Retorna prioritat del fil
- **setPriority(int p)** → Canvia la prioritat a p

# Mètodes útils en fils

- **void interrupt()** → Interromp execució fil
- **boolean interrupted()** → Comprova si fil actual ha estat interromput.
- **Thread currentThread()** → Retorna referència a l'objecte fil que s'està executant actualment.

# Exemple T2S2Exemple7

FilExemple2.java

```
1  public class FilExemple2 extends Thread {
2      //Mètode run
3      public void run(){
4      System.out.println ("Dins del Fil: " + this.getName() + " Prioritat: " + this.getPriority() + " ID: " + this.getId());
5      }//fi run
6
7      //Mètode main
8      public static void main (String[] args){
9
10      FilExemple2 h=null;
11
12     for (int i=0;i<3; i++){
13             h= new FilExemple2(); //cree fil
14
15             h.setName ("FIL " + i); //establisc nom
16             h.setPriority (i+1); //establisc prioritat
17
18             h.start(); //inicie fil
19
20             System.out.println ("Informació del " + h.getName() + ": " + h.toString());
21         }
22     System.out.println ("3 Fils creats...");
23
24     }//fi main
25
26 }//fi classe
```

# Exemple T2S2Exemple7

**Atenció: l'eixida pot variar d'una execució a una altra**

```
[ramon@nowhere Exemples]$ javac FilExemple2.java
[ramon@nowhere Exemples]$ java FilExemple2
Informació del FIL 0: Thread[FIL 0,1,main]
Informació del FIL 1: Thread[FIL 1,2,main]
Informació del FIL 2: Thread[FIL 2,3,main]
3 Fils creats...
Dins del Fil: FIL 1 Prioritat: 2 ID: 12
Dins del Fil: FIL 0 Prioritat: 1 ID: 11
Dins del Fil: FIL 2 Prioritat: 3 ID: 13
```

# Activitat T2S2P1TicTac

Crea un programa que mostre per pantalla TICTACTICTAC i així fins a l'infinit. Ha de tindre la mateixa cadència, és a dir, per exemple entre el TIC i el TAC ha de passar un segon i entre el TAC i el TIC, un altre segon més.

Per a això, has de tindre un main que cride a dos fils, un per a TIC i un altre per a TAC. Aquests dos fils han d'executar-se fins a l'infinit.

Pots usar el mètode sleep.

**Es visualitza TIC TAC de forma ordenada (és a dir TICTACTICTAC)?**